

VITyarthi

Project Report

Expense Tracker & Analyzer

Course	Python Essentials
Student Name	Aman Kumar
Registration No.	26BCE10228
GitHub Repository	[https://github.com/amankumar-0977/expense-tracker]
Submission Date	30/09/2026

Table of Contents

1. CoverPage
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Managing personal finances begins with knowing where money is spent. This project, **Expense Tracker & Analyzer**, is a modular command-line application written in Python that lets a user record expenses, define monthly budgets per category, and analyse spending through statistical reports and trends.

The project applies the concepts covered in the Python Essentials course: variables and operators, input/output, data structures (lists, dictionaries, tuples), control flow, functions, modules and packages, exception handling, file handling and object-oriented programming. The application uses only the Python standard library, so it runs on any machine with Python installed and needs no internet connection.

The code is organised in layers (presentation, service, storage and utilities), which keeps each file small,

3. Problem Statement

People who track spending on paper, in memory or in scattered notes find it hard to know how much they have spent, in which category, and whether they are within their limits. Spreadsheets are flexible but slow for quick daily entry, and many mobile apps require accounts, internet access or store financial data online.

There is a need for a **simple, offline and fast tool** that records expenses, warns about overspending and converts raw entries into meaningful statistics.

Objectives

- Provide complete CRUD operations on expense records with validation.
- Allow category-wise monthly budgets with automatic overspending alerts.
- Generate a monthly statistical report and a month-by-month spending trend.
- Persist data locally (JSON) and support CSV export.
- Handle every kind of bad input gracefully without crashing.
- Follow a modular, well-documented structure and use Git for version control.

Target users: students, young professionals and anyone who wants a private, offline expense log.

4. Functional Requirements

The system has three major functional modules: **Expense Management**, **Budget Management** and **Analytics & Reporting**, supported by a data export feature.

ID	Module	Requirement
FR1	Expense Mgmt	Add an expense with amount, category, optional description, date (default: today) and recurring flag with frequency (weekly / monthly / yearly).
FR2	Expense Mgmt	View all expenses or filter by month (YYYY-MM); show record count and total.
FR3	Expense Mgmt	Edit an expense by id; blank fields keep the existing value.
FR4	Expense Mgmt	Delete an expense by id after a confirmation prompt.
FR5	Expense Mgmt	Search by keyword, category and minimum / maximum amount.
FR6	Budget Mgmt	Set a monthly spending limit per category.
FR7	Budget Mgmt	Show budget status for a month: amount spent vs limit, percentage used, progress bar and status.
FR8	Budget Mgmt	Print an alert immediately after adding an expense that pushes a category past its budget.
FR9	Analytics	Monthly report: count, total, mean, median, min, max, standard deviation, largest expense and category breakdown with percentages.
FR10	Analytics	Spending trend across all months with a scaled text bar chart.
FR11	Data	Export all expenses to a CSV file.
FR12	Data	Load and save data automatically in a JSON file; allow a custom file with --data.

Input / Output structure. Input: menu choice and prompted values typed at the keyboard, plus the optional -data argument. Output: formatted tables, reports and progress bars on the terminal, the JSON data file, an optional CSV file and a log file.

User workflow. Start the program, choose an option from the menu, answer the prompts, view the result, and return to the menu until 0 (Exit) is chosen (see Figure 3).

5. Non-Functional Requirements

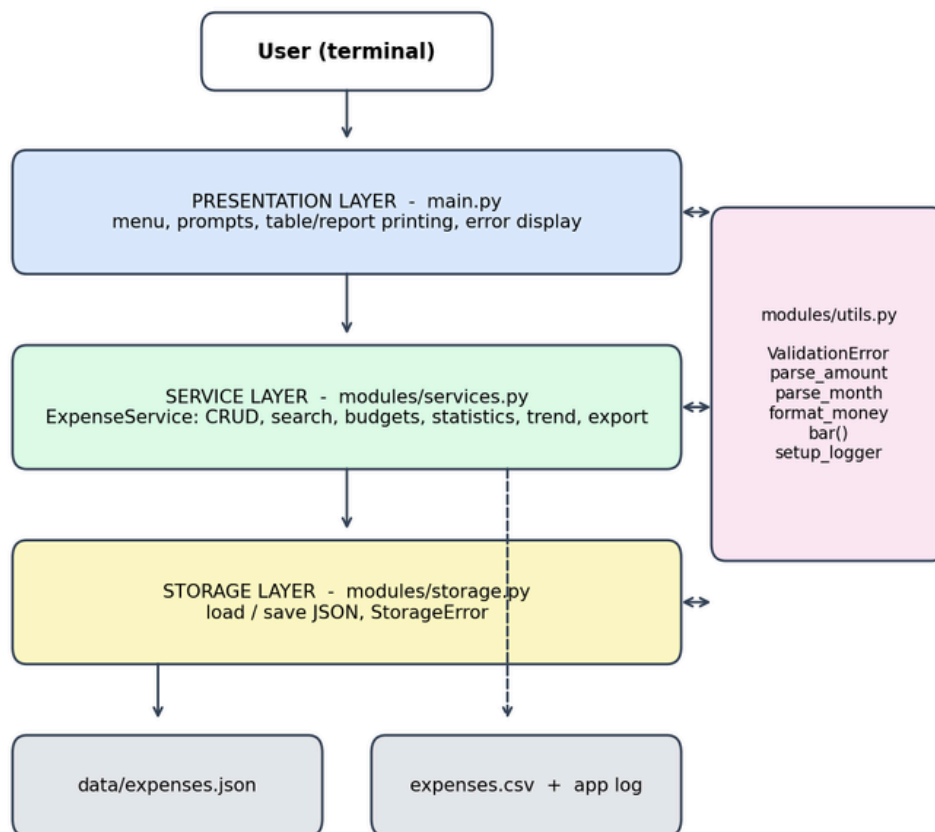
ID	Category	Requirement and how it is met
NFR1	Usability	Numbered menu, clear prompts with examples (e.g. 'Date YYYY-MM-DD'), blank-input defaults and readable aligned tables.
NFR2	Reliability / Error handling	Custom exceptions (ValidationError, NotFoundError, StorageError) are caught in the main loop; bad input shows a friendly message and the program continues. A last-resort handler catches unexpected errors.
NFR3	Maintainability	Layered design with separate files for CLI, business logic, storage and utilities; small single-purpose functions; comments and docstrings.
NFR4	Logging / Monitoring	Application start, close and unexpected exceptions are logged through Python's logging module.
NFR5	Security / Privacy	Data stays on the user's machine in a local file; no network access; all input is validated and sanitised before use.
NFR6	Performance / Resource efficiency	Data is small and held in memory as lists; operations are O(n). Only the standard library is used, so memory and disk use are minimal.
NFR7	Portability	Pure Python 3 with no third-party packages; works on Windows, macOS and Linux.
NFR8	Scalability	Storage is isolated in one module, so JSON can be replaced by SQLite without touching the CLI or service layer.

6. System Architecture

The application follows a **layered architecture**. Each layer talks only to the layer directly below it, and shared helpers live in a utility module.

Layer	File	Responsibility
Presentation	main.py	Menu loop, prompts, argument parsing, printing tables/reports, and displaying friendly errors.
Service (business logic)	modules/services.py	ExpenseService: CRUD, search, budgets, alerts, statistics, trend and CSV export.
Storage	modules/storage.py	Reading and writing the JSON data file; raises StorageError.
Utilities	modules/utls.py	Validation, parsing (amount, month), money formatting, text progress bar, logger setup.
Models	modules/models.py	Expense and Budget classes holding the data.

Figure 1: Layered System Architecture

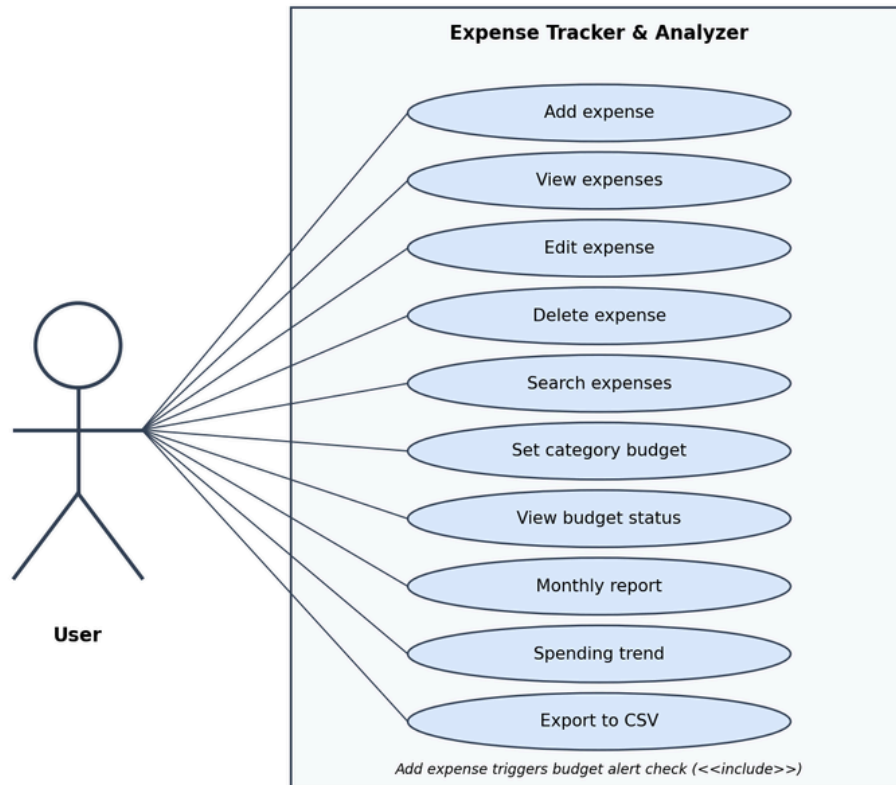


7. Design Diagrams

7.1 Use Case Diagram

The single actor (User) interacts with ten use cases. Adding an expense automatically includes a budget alert check.

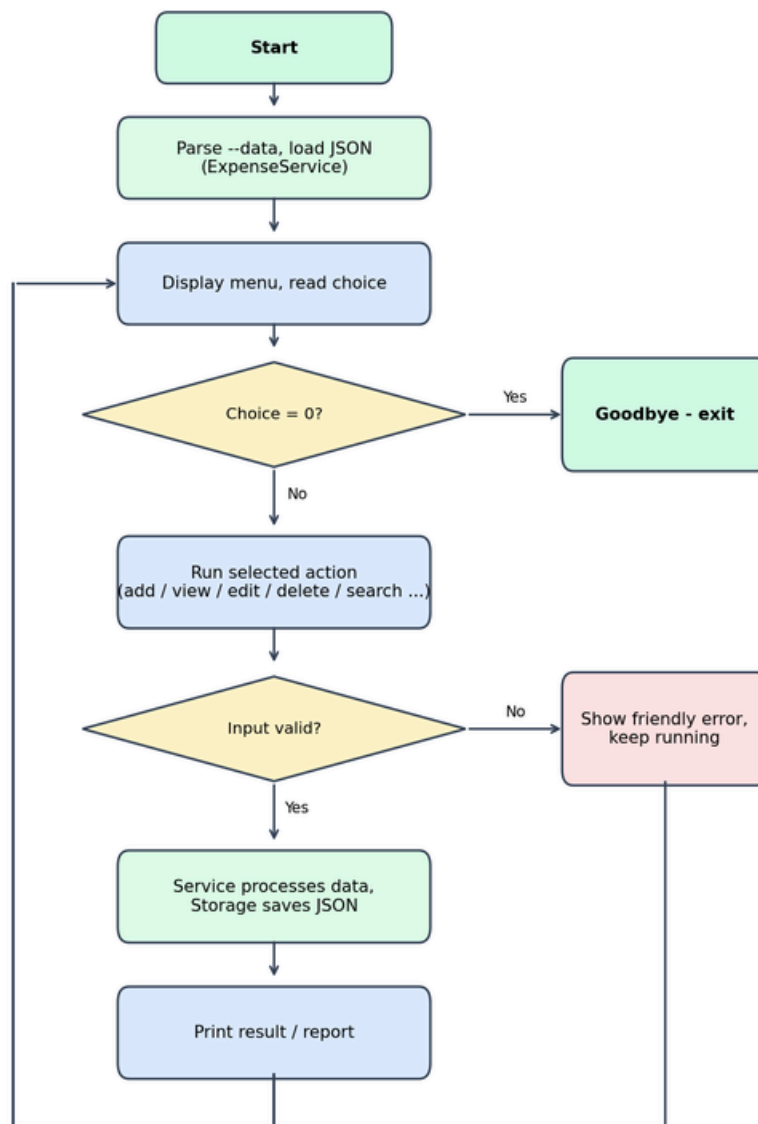
Figure 2: Use Case Diagram



7.2 Workflow Diagram

The program loops until the user chooses 0. Errors never end the loop; they only display a message.

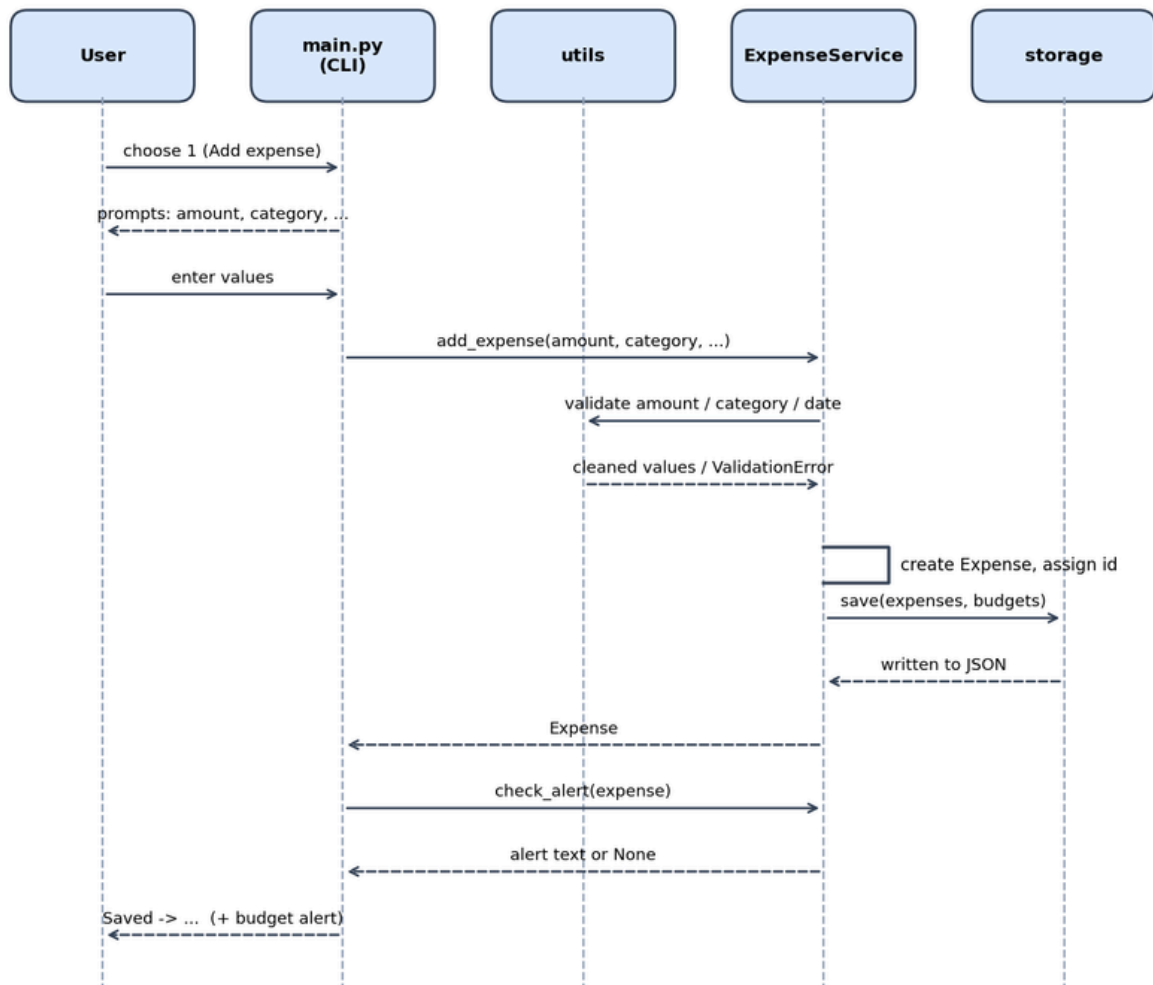
Figure 3: Application Workflow



7.3 Sequence Diagram

Sequence of messages when the user adds an expense.

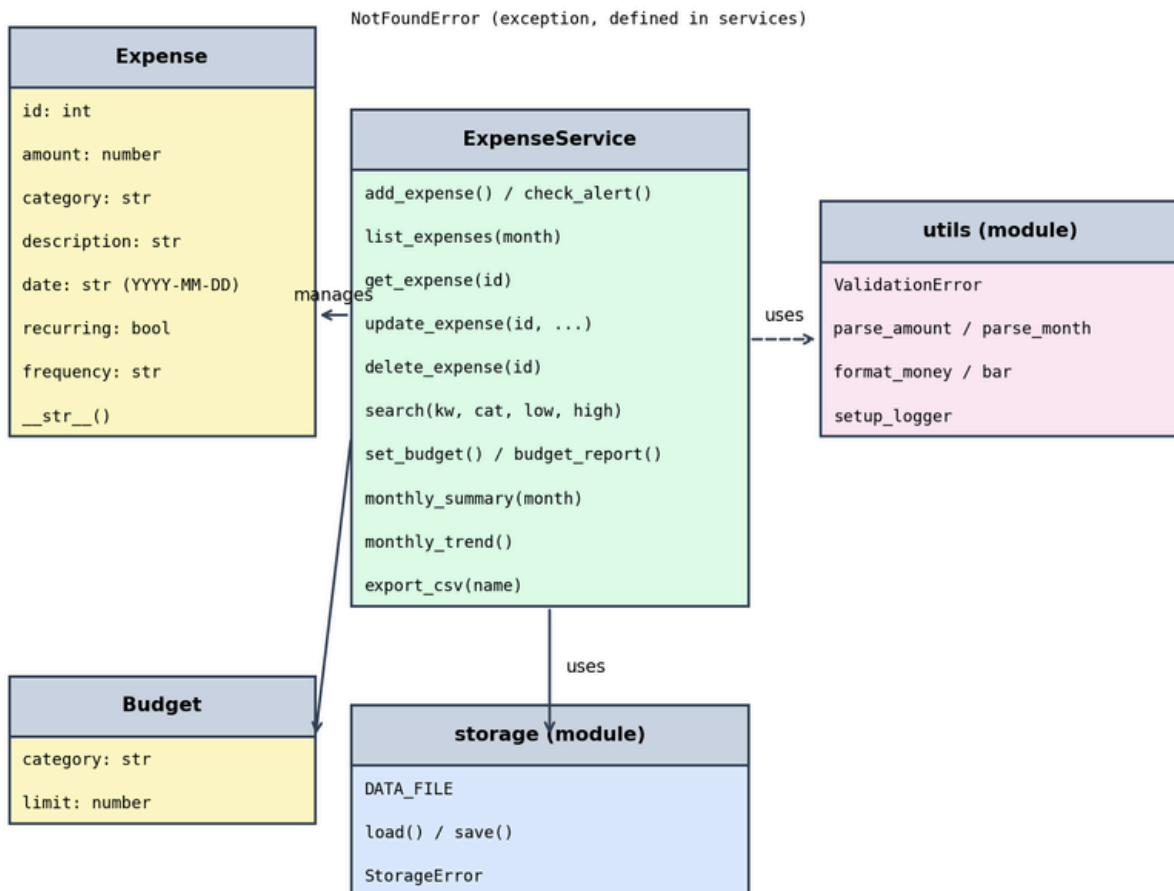
Figure 4: Sequence Diagram - Add Expense



7.4 Class / Component Diagram

ExpenseService manages Expense and Budget objects, persists them through the storage module and relies on utils for validation and formatting.

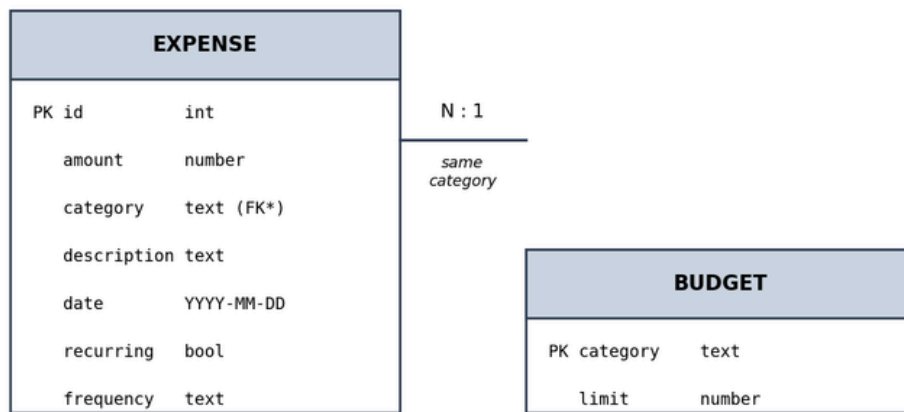
Figure 5: Class / Component Diagram



7.5 ER Diagram and Storage Design

Data is stored as one JSON file with two collections. Expense and Budget are linked logically by the category name.

Figure 6: ER Diagram (JSON storage schema)



* logical link by category name (not an enforced foreign key); a category may have expenses but no budget.

```
{
  "expenses": [ { "id": 1, "amount": 250, "category": "Food",
                  "description": "Lunch", "date": "2026-09-28",
                  "recurring": false, "frequency": "monthly" } ],
  "budgets": [ { "category": "Food", "limit": 3000 } ]
}
```

Note: the exact JSON key names follow the implementation in storage.py; the above is representative.

8. Design Decisions & Rationale

Decision	Rationale
Layered, multi-file structure	Separates user interface from logic and storage, which makes each part testable and replaceable, and meets the modular-design expectation.
JSON for storage	Human-readable, built into Python, needs no server and is easy to back up or edit. A database would be excessive for a single-user tool.
Custom exception classes	Let the main loop distinguish bad input (ValidationError), missing records (NotFoundError) and file problems (StorageError) and respond appropriately.
Action dispatch dictionary (ACTIONS)	Maps menu choices to functions; adding a feature needs one function and one dictionary entry, not a long if/elif chain.
Text progress bars	Give quick visual feedback for budgets and trends with no plotting library, keeping the app dependency-free.
Standard library statistics	Mean, median and standard deviation are computed with the built-in statistics module for correctness.
Blank input = default	Reduces typing (today's date, current month, keep old value on edit), improving usability.
Confirmation before delete	Prevents accidental data loss.
Central logger	Oneplace to record events and unexpected errors for debugging.

9. Implementation Details

main.py builds the menu, reads input through the helper functions `get_input()` and `get_number()`, and calls the service. The `main()` function parses the `--data` argument, creates an `ExpenseService` (a `StorageError` at startup is reported and the program exits cleanly), and runs the menu loop. Inside the loop, every action runs in a `try/except` that catches `ValidationError`, `NotFoundError` and `StorageError` (friendly message), `EOFError` and `KeyboardInterrupt` (clean exit), and any other `Exception` (logged with a traceback).

Menu actions. Each menu option is a small function taking the service object: `add_expense`, `view_expenses`, `edit_expense`, `delete_expense`, `search_expenses`, `set_budget`, `monthly_report`, `spending_trend` and `export_csv`. Output is formatted with f-string alignment (for example `{category:<14} {amount:>10}`) so tables line up.

Analytics. `monthly_summary` returns a dictionary containing count, total, mean, median, min, max, standard deviation, the largest expense and a breakdown list of (category, total, percent). The report prints this dictionary. `monthly_trend` returns (month, total) pairs; the trend view scales each bar to the peak month.

Python concepts applied

Concept	Where used
Functions and modules	Ten menu functions in <code>main.py</code> ; imports from the <code>modules</code> package
Data structures	Lists of <code>Expense</code> objects, dictionaries for summaries and <code>ACTIONS</code> , tuples for report rows and trend data
Control flow	Main while-loop, if/else menu handling, for-loops over records
OOP	<code>Expense</code> and <code>Budget</code> classes; <code>ExpenseService</code> class; custom exception classes
Exception handling	Custom exceptions and <code>try/except</code> in the main loop
File handling	JSON load/save, CSV export, log file
String formatting	f-strings with width and alignment for tables and reports

Concept	Where used
Command-line arguments	argparse for --data

10. Screenshots / Results

The listings below show the format produced by the application (values are illustrative). Replace or supplement them with real screenshots from your run, stored in the screenshots/ folder.

Main menu

```
===== EXPENSE TRACKER =====
1. Add expense
2. View expenses
3. Edit expense
4. Delete expense
5. Search expenses
6. Set category budget
7. Budget status
8. Monthly report
9. Spending trend
10. Export to CSV
0. Exit
=====
```

Adding an expense with a budget alert

```
Choose an option: 1
Amount: 600
Category (e.g. Food, Travel): Food
Description (optional): Dinner
Date YYYY-MM-DD (blank = today):
Recurring? (y/N): n
Saved -> 7 2026-09-28 Food 600 Dinner
*** <budget alert message> ***
```

Viewing expenses

ID	Date	Category	Amount	Description
1	2026-09-02	Food	250	Lunch
2	2026-09-05	Travel	120	Bus pass

2 record(s), total 370

Monthly report

```
-----Report for 2026-09 -----
Transactions : 2
Total spent : 370
Average      : 185   Median: 185
Min / Max    : 120 / 250
Std. dev.    : 91.9
Largest      : <largest expense>
```

```
By category:
Food          250   67.6%  [#####...]
Travel        120   32.4%  [###.....]
```

Real outputs may differ slightly in currency symbol and bar characters, depending on `format_money()` and `bar()` in `utils.py`.

11. Testing Approach

The project is tested with **validation (black-box) tests**: each test gives a specific input through the menu and checks the observed behaviour against the expected result. Tests were run manually against a fresh data file, using `python main.py --data test.json` so that real data is never touched.

ID	Test case	Input	Expected result
T1	Add valid expense	1, 250, Food, Lunch, blank date, n	Expense saved with today's date and a new id

ID	Test case	Input	Expected result
T2	Non-numeric amount	1, abc	'Please enter a whole number'; menu returns
T3	Invalid menu option	99 or text	'Invalid choice' message
T4	Invalid date	Date 2026-13-45	Validation error; nothing saved
T5	Empty category	Category left blank	Validation error
T6	Edit with blanks	3, valid id, all fields blank	Record unchanged
T7	Unknown id	3 or 4 with id 9999	Not-found error; no crash
T8	Delete cancelled	4, valid id, N	'Cancelled'; record remains
T9	Search filters	5 with keyword, category and amount range	Only matching records shown
T10	Budget alert	Budget Food=500, then add Food 600	Alert printed after save
T11	Budget status	Option 7 with no budgets	'No budgets set yet' message
T12	Empty month report	Option 8 for month without data	'No expenses recorded' message
T13	Persistence	Add, exit, restart, view	Data still present
T14	Corrupt data file	Invalid JSON in data file	'Startup problem' message, clean exit
T15	CSV export	Option 10, blank name	expenses.csv created and opens in a spreadsheet
T16	Ctrl+C / Ctrl+D	Interrupt at a prompt	Clean exit, no traceback

Record the actual pass/fail result for each test in your own run and attach screenshots of a few key cases.

12. Challenges Faced

Challenge	How it was addressed
Keeping the program alive on bad input	Defined custom exceptions and caught them in one place in the main loop instead of scattering try/except in every function.
Mixing display code with business logic	Moved all calculations to ExpenseService and kept main.py limited to input and printing.
Aligned, readable terminal output	Used f-string width and alignment specifiers and a text progress bar helper.
Handling blank input in edit/search	Treated blank as 'keep' or 'ignore this filter' and converted only non-blank values.
Date and month validation	Centralised parsing in utils so every feature accepts the same formats and gives the same error messages.
Protecting stored data	Isolated file access in storage.py with its own StorageError, and reported startup file problems cleanly.

13. Learnings & Key Takeaways

- Designing in layers makes a program easier to extend, debug and explain.
- Custom exceptions give precise, user-friendly error messages and keep control flow clean.
- A dispatch dictionary is a neat replacement for long if/elif chains.
- The standard library (json, csv, statistics, logging, argparse) is powerful enough for a complete application.
- Testing edge cases such as empty data, unknown ids and corrupt files reveals bugs that normal use never shows.
- Regular Git commits document progress and make mistakes easy to undo.

14. Future Enhancements

- Automatically generate recurring expenses on their due date.
- Accept decimal amounts throughout (currently a whole number is requested when adding).
- Add SQLite storage and user accounts for multi-user use.
- Graphical charts with matplotlib, and a Tkinter or web (Flask) interface.
- Import from bank statements (CSV) and support multiple currencies.
- Add automated unit tests (unittest / pytest) and a continuous integration workflow.
- Monthly email or notification summaries and savings goals.

15. References

- Python Software Foundation. Python 3 Documentation - json, csv, statistics, logging and argparse modules. <https://docs.python.org/3/>
- VITyarthi. Python Essentials course material and Build Your Own Project guidelines.
- Python Software Foundation. PEP 8 - Style Guide for Python Code. <https://peps.python.org/pep-0008/>
- Pro Git Book. <https://git-scm.com/book>